

Documentação Complementar

Sistema Inteligente de Gerenciamento da Infraestrutura da Colônia (SIGIC)

Colônia Marciana Aurora Siger

1. Introdução

Este documento descreve em detalhes a arquitetura, as decisões de design, os algoritmos implementados e as justificativas técnicas do **SIGIC — Sistema Inteligente de Gerenciamento da Infraestrutura da Colônia**. O sistema foi desenvolvido em Python e representa computacionalmente a rede operacional da colônia marciana **Aurora Siger**, integrando algoritmos de grafos, modelagem matemática, estruturas de dados e análise de sustentabilidade (ESG).

2. Organização da Infraestrutura da Colônia

2.1 Módulos da Colônia

A Aurora Siger é composta por **8 módulos essenciais**, definidos na constante `NOMES_BASE`. Cada módulo possui atributos gerados dinamicamente pela função `gerar_dados_aleatorios()` e armazenados como **tuplas** no dicionário `modulos_info`:

Módulo	Função Principal	Prioridade Operacional
Habitação	Acomodação e suporte básico à vida da tripulação	Alta
Centro de Controle	Monitoramento e gerenciamento das operações da base	Alta
Armazenamento de Energia	Armazenamento da energia produzida pelos sistemas da base	Crítica
Agricultura	Produção de alimentos e sustentabilidade da colônia	Média
Laboratório Científico	Pesquisas e análises de materiais e condições marcianas	Baixa
Comunicação	Troca de dados entre módulos e comunicação com a Terra	Média
Suporte Médico	Atendimento médico e monitoramento da saúde da tripulação	Alta
Produção de Oxigênio	Geração e distribuição de oxigênio para a base	Crítica

2.2 Atributos dos Módulos

Cada módulo possui os seguintes atributos, representados como uma **tupla de 5 elementos** dentro do dicionário `modulos_info`:

Índice	Atributo	Tipo	Intervalo Simulado	Descrição
[0]	Consumo energético	int	20–100 kW	Energia necessária para o funcionamento do módulo
[1]	Prioridade operacional	int	1–3	Nível de importância (1 = mais crítico)
[2]	Capacidade de armazenamento	int	50–500 kW	Capacidade de armazenar recursos ou energia
[3]	Status operacional	str	Ativo / Manutenção / Alerta	Situação atual do módulo
[4]	Necessidade de comunicação	str	Alta / Média / Baixa	Frequência de troca de informações

Os dados são **regenerados a cada simulação** (Opção 5 do menu), mas a topologia da rede permanece fixa, separando o estado dinâmico das propriedades estruturais do grafo.

3. Representação da Rede com Grafos

3.1 Topologia da Rede

A infraestrutura da Aurora Siger é modelada como um **grafo não-direcionado ponderado**, em que:

- **Vértices** = módulos da colônia
- **Arestas** = conexões físicas/energéticas entre módulos
- **Pesos** = distância entre os módulos (em unidades arbitrárias), usada como proxy de custo energético de transmissão

As 10 conexões definidas em `conexoes_lista` são:

```
Armazenamento de Energia <--(5)--> Centro de Controle
Centro de Controle        <--(10)--> Habitação
Centro de Controle        <--(8)--> Comunicação
Habitação                <--(5)--> Suporte Médico
Habitação                <--(12)--> Produção de Oxigênio
Produção de Oxigênio      <--(15)--> Agricultura
Agricultura               <--(20)--> Laboratório Científico
Armazenamento de Energia <--(10)--> Produção de Oxigênio
Comunicação               <--(15)--> Laboratório Científico
Suporte Médico            <--(10)--> Laboratório Científico
```

3.2 Justificativa da Topologia

As conexões **não foram escolhidas aleatoriamente**; cada aresta reflete uma necessidade operacional real de uma colônia marciana:

Armazenamento de Energia como hub central: é o módulo mais conectado (liga-se ao Centro de Controle e à Produção de Oxigênio com os menores pesos), pois é a fonte primária de energia e precisa alimentar diretamente os módulos de maior criticidade com o mínimo de perda.

Centro de Controle como hub de coordenação: interliga Habitação e Comunicação, monitorando tanto o suporte à vida quanto a troca de dados com a Terra. Sua conexão com o Armazenamento de Energia garante controle direto sobre os recursos energéticos.

Suporte Médico e Produção de Oxigênio próximos à Habitação: sistemas de suporte à vida precisam de baixa latência de resposta. A distância curta (peso 5) entre Habitação e Suporte Médico reflete essa urgência.

Laboratório Científico na periferia: conectado apenas a Agricultura, Comunicação e Suporte Médico, com pesos elevados (15–20). Isso reflete sua menor prioridade operacional imediata — consome dados em vez de demandar resposta rápida.

3.3 Representações Matricial e de Lista

O sistema mantém **duas representações simultâneas** do grafo para finalidades distintas:

Lista de Adjacência (`lista_adj`): dicionário Python onde cada chave é um módulo e o valor é uma lista de tuplas (`vizinho`, `peso`). Construída pela função `construir_lista_adjacencia()`. É a estrutura primária usada pelos algoritmos BFS, DFS e Dijkstra, pois oferece acesso eficiente aos vizinhos de cada nó.

Matriz de Adjacência (`matriz_adj`): array NumPy 8×8 simétrico construído por `inicializar_matriz_adjacencia()`. Os valores representam os pesos das arestas (0 onde não há conexão). Utilizada para visualização matemática da rede (Opção 2 do menu) e operações matriciais.

4. Estruturas de Dados em Python

4.1 Justificativa das Escolhas

Estrutura	Onde é Usada no Código	Justificativa
Dicionário	<code>modulos_info</code> , <code>lista_adj</code> , <code>distancias</code> , <code>predecessores</code>	Acesso O(1) por nome do módulo; chave semântica torna o código legível e evita erros de índice numérico
Lista	<code>NOMES_BASE</code> , <code>conexoes_lista</code> , <code>caminho</code> , <code>fila</code> (BFS)	Coleções ordenadas e mutáveis; essencial para manter a sequência de módulos e reconstruir caminhos
Tupla	Arestas em <code>conexoes_lista</code> (origem, destino, peso); atributos em <code>modulos_info</code>	Imutabilidade garante integridade dos dados de configuração da rede; uso de índice fixo por posição semântica

Estrutura	Onde é Usada no Código	Justificativa
Matriz NumPy	<code>matriz_adj</code>	Representação matemática compacta; viabiliza operações matriciais e exibição tabular via Pandas
Fila de Prioridade (heapq)	Algoritmo de Dijkstra (<code>pq</code>)	Extração eficiente do menor custo acumulado em $O(\log n)$, essencial para a correteude e desempenho do Dijkstra
Conjunto (set)	<code>visitados</code> em <code>bfs_caminho</code>	Verificação de pertencimento em $O(1)$, eliminando o gargalo de busca em lista na detecção de nós visitados
Counter	<code>realizar_analise_esg()</code>	Contagem segura e concisa de status dos módulos, evitando <code>KeyError</code> para categorias com zero ocorrências

4.2 Decisão de Design: Dados Dinâmicos vs. Topologia Fixa

Uma escolha deliberada do projeto foi **separar os dados dos módulos (dinâmicos) da topologia da rede (fixa)**. Os atributos em `modulos_info` são regenerados a cada simulação, enquanto `conexoes_lista` e as estruturas derivadas (`lista_adj`, `matriz_adj`) são calculadas uma única vez na inicialização. Isso reflete a realidade de uma colônia: os módulos variam em consumo e status continuamente, mas a infraestrutura física de conexões é um investimento permanente.

5. Algoritmos de Redes Implementados

5.1 Busca em Largura — BFS

Função: `bfs(inicio)` e `bfs_caminho(inicio, fim)`

A BFS explora a rede nível a nível a partir do nó inicial, usando uma **fila** (FIFO). Garante que o caminho encontrado tenha o **menor número de saltos** (arestas), independentemente dos pesos.

No SIGIC, a BFS tem dois usos distintos:

- `bfs(inicio)`: retorna a ordem de visitação de todos os módulos alcançáveis. Usada na detecção de componentes conectados após simulação de falha.
- `bfs_caminho(inicio, fim)`: versão adaptada que rastreia predecessores para reconstruir a rota real com menor número de saltos. Usada na Opção 3 para comparar com a rota do Dijkstra, evidenciando que "menos saltos" não significa "menor custo energético".

Complexidade: $O(V + E)$, onde V = número de módulos, E = número de conexões.

5.2 Busca em Profundidade — DFS

Função: `dfs(inicio, visitados=None)` e `dfs_pontes(u)` (interna)

A DFS explora o grafo seguindo um caminho até o seu fim antes de retroceder, usando **recursão** (pilha implícita). No SIGIC:

- `dfs(inicio)`: exploração geral da rede, retornando a sequência de visitação em profundidade.
- `dfs_pontes(u)`: DFS especializada que mantém **tempos de descoberta** (`descoberta[]`) e **valores low** (`low[]`) para identificar pontes pelo algoritmo de Tarjan. Um nó `v` é ponte de `u` quando `low[v] > descoberta[u]`, ou seja, não há caminho alternativo que alcance `u` ou seus ancestrais sem passar pela aresta `(u, v)`.

Complexidade: $O(V + E)$.

5.3 Algoritmo de Dijkstra

Função: `dijkstra(inicio, fim)`

Encontra o **caminho de menor custo total** entre dois módulos, considerando os pesos das arestas.

Implementado com **fila de prioridade mínima** (`heapq`) para eficiência:

1. Inicializa todas as distâncias como infinito, exceto a origem (distância 0).
2. Extrai o nó de menor distância acumulada da fila.
3. Relaxa as arestas dos vizinhos: se o novo custo for menor, atualiza e insere na fila.
4. Rastreia predecessores para reconstrução do caminho.

No SIGIC, a distância calculada pelo Dijkstra serve diretamente para estimar a **perda energética** na transmissão entre módulos, usando a constante `TAXA_PERDA_ENERGETICA = 0.02` (2% por unidade de distância):

```
Perda estimada = distancia_mínima × 0.02 [kW]
```

Complexidade: $O((V + E) \log V)$ com heap binário.

5.4 Detecção de Pontes (Conexões Críticas)

Função: `encontrar_pontes()`

Implementa o **algoritmo de Tarjan** para identificação de pontes, que são arestas cuja remoção desconecta o grafo. Na Aurora Siger, uma ponte representa uma **conexão única de falha**: se essa ligação física falhar, parte da colônia ficará sem comunicação ou energia.

O algoritmo mantém dois arrays:

- `descoberta[u]`: tempo em que o nó `u` foi visitado pela primeira vez.
- `low[u]`: menor tempo de descoberta alcançável a partir da subárvore de `u`.

Uma aresta `(u, v)` é ponte se `low[v] > descoberta[u]`.

Essa detecção é integrada à simulação de falhas (Opção 10): ao remover uma conexão, o sistema verifica se ela era uma ponte na rede original e alerta o operador.

6. Modelagem Matemática e Otimização

6.1 Modelo de Consumo Energético

Função: `modelagem_energetica()`

O sistema modela o crescimento do consumo energético da colônia ao longo do tempo usando uma **função quadrática**:

$$f(t) = at^2 + bt + c$$

onde:

- **t** = tempo em meses a partir do início da projeção
- **a, b, c** = coeficientes calculados por regressão polinomial (`numpy.polyfit` de grau 2) sobre 5 pontos de dados históricos simulados
- **f(t)** = consumo energético total da colônia no mês t (em kW)

6.2 Geração dos Dados Históricos

O histórico simulado é gerado com tendência quadrática intencional:

```
consumo_simulado = base_consumo + (i2) × fator_aleatório + ruído
```

O último ponto é ancorado ao consumo real atual (`sum(v[0] for v in modulos_info.values())`), garantindo que a projeção seja realista em relação ao estado presente da colônia.

6.3 Análise Diferencial

A derivada da função de consumo é calculada simbolicamente com **SymPy**:

$$f'(t) = 2at + b$$

Representa a **taxa de variação do consumo** por mês. No mês atual (**t = 5**):

- Se $f'(5) > 0$: consumo crescendo — expansão da colônia em curso.
- Se $f'(5) < 0$: consumo declinando — possível redução de operações ou ganho de eficiência.

6.4 Análise Qualitativa da Concavidade

O coeficiente **a** determina o comportamento de longo prazo:

Sinal de a	Interpretação para a colônia
$a > 0$	Crescimento acelerado — a demanda energética aumenta cada vez mais rápido; expansão da infraestrutura é urgente
$a < 0$	Crescimento desacelerado — o consumo tende a atingir um máximo e recuar; modelo de estabilização
$a \approx 0$	Comportamento aproximadamente linear — crescimento estável e previsível

6.5 Previsão de Saturação

O sistema resolve a equação $f(t) = C_{total}$ com SymPy, onde C_{total} é a capacidade total de armazenamento da colônia (`calcular_capacidade_total()`):

$$at^2 + bt + c = C_{total}$$

A primeira raiz real positiva indica o **mês em que o consumo atingirá o limite máximo de armazenamento**, alertando para a necessidade de expansão ou otimização antes desse horizonte.

Limitação do modelo: por ser ajustado a apenas 5 pontos simulados, representa uma tendência de curto prazo. Eventos externos (novas missões, falhas, expansões) podem alterar significativamente o comportamento real.

7. Simulação de Falhas na Rede

7.1 Funcionamento

Função: `simular_falha_rede()`

Permite testar dois tipos de falha:

Falha de Módulo: remove o módulo selecionado de `nomes_modulos` e `modulos_info`, eliminando também todas as arestas que o conectam. Simula a perda completa de um setor da colônia.

Falha de Conexão: remove uma aresta específica de `conexoes_lista`. Simula a ruptura de uma ligação física (cabo, duto, corredor pressurizado).

7.2 Análise de Impacto Automatizada

Após a falha simulada, o sistema realiza automaticamente:

1. **Deteção de particionamento:** aplica BFS a partir de cada módulo remanescente para identificar componentes conectados. Se houver mais de um componente, a rede foi particionada e módulos ficaram isolados.
2. **Verificação de criticidade da conexão removida:** compara a aresta removida com a lista de pontes da rede original para determinar se era uma conexão crítica.
3. **Recálculo de rotas Dijkstra:** demonstra o impacto na rota mais eficiente entre dois módulos após a falha.

7.3 Mecanismo de Backup e Restauração

Antes de qualquer modificação, o estado completo da rede é salvo em variáveis locais (backup profundo de listas e dicionários). Ao final da simulação, o estado original é **restaurado automaticamente**, garantindo que múltiplas simulações possam ser realizadas na mesma sessão sem corromper os dados.

8. Análise de Eficiência Operacional e Governança ESG

8.1 Métricas Calculadas

Função: `realizar_analise_esg()`

Métrica	Cálculo	Finalidade
Consumo total	<code>sum(v[0] for v in modulos_info.values())</code>	Avalia a demanda energética atual da colônia
Capacidade total	<code>sum(v[2] for v in modulos_info.values())</code>	Determina a margem disponível antes da saturação
% em Alerta	<code>(contagem_alerta / 8) × 100</code>	Sinaliza instabilidade operacional generalizada
% em Manutenção	<code>(contagem_manutencao / 8) × 100</code>	Indica necessidade de reforço de equipes técnicas
Ranking custo/prioridade	<code>consumo / prioridade</code> por módulo	Identifica módulos com alto consumo relativo à sua importância
Módulos críticos	<code>P1 + consumo > 70 kW + capacidade < 150 kW</code>	Detecta sistemas de suporte à vida sobrecarregados

8.2 Recomendações de Governança Dinâmica

O sistema gera recomendações automáticas baseadas no estado atual, seguindo esta lógica:

Se % em manutenção > 15%

→

Reforçar equipes de manutenção preventiva

Se % em alerta > 10%

→

Revisar protocolos e mitigar riscos em módulos críticos

Caso contrário

→

Focar em expansão controlada e redundância de rede

Módulos críticos detectados geram um **alerta de governança explícito**: o sistema deixa claro que a sinalização é de apoio à decisão e que nenhuma ação é executada automaticamente. Qualquer realocação de energia ou prioridade de manutenção deve ser revisada e aprovada por um responsável humano.

9. Interface do Sistema (Menu Terminal)

O sistema oferece um menu de 11 opções via terminal com navegação numérica:

Opção	Funcionalidade	Algoritmo/Estrutura Principal
1	Visualizar infraestrutura (tabela de módulos)	DataFrame Pandas
2	Matriz de adjacência da rede	NumPy + Pandas
3	Rota otimizada (Dijkstra) com comparação BFS	Dijkstra + BFS
4	Exploração da rede por BFS ou DFS	BFS / DFS
5	Simular novo cenário operacional (dados aleatórios)	<code>random</code> , dicionários

Opção	Funcionalidade	Algoritmo/Estrutura Principal
6	Modelagem matemática e previsão de saturação	NumPy polyfit + SymPy
7	Análise ESG e governança	Counter, ranking
8	Detectar conexões críticas (pontes)	Algoritmo de Tarjan
9	Consultar módulo individual	Dicionário + lista de adjacência
10	Simular falha de módulo ou conexão	BFS + Dijkstra + Tarjan
11	Visualizar conexões em formato legível	Lista de tuplas

A função `selecionar_modulo()` substitui a digitação manual de nomes por seleção numérica, eliminando erros de acentuação e tornando a navegação mais robusta.

10. Sustentabilidade e Expansão da Colônia

10.1 Uso Sustentável de Energia

A constante `TAXA_PERDA_ENERGETICA = 0.02` incorpora no cálculo de rotas a **perda energética por distância**, incentivando o uso do caminho de menor custo real (Dijkstra) em vez de rotas com menos conexões intermediárias (BFS). Isso reflete a física da transmissão energética em longas distâncias.

10.2 Expansão Organizada

A separação entre topologia fixa e dados dinâmicos permite que novos módulos sejam adicionados a `NOMES_BASE` e novas conexões a `conexoes_lista` sem alterar os algoritmos. As funções `construir_lista_adjacencia()` e `inicializar_matriz_adjacencia()` reconstroem automaticamente as estruturas derivadas.

10.3 Priorização de Sistemas Críticos

O campo de prioridade (1 = crítico, 3 = baixa prioridade) é usado na análise ESG para calcular o **custo energético por unidade de prioridade** (`consumo / prioridade`), auxiliando decisões de corte de energia em emergências: módulos com razão alta consomem muito em relação à sua importância e são candidatos prioritários à redução de carga.

10.4 Redundância e Pontos de Falha

A detecção de pontes (Opção 8) é a principal ferramenta de planejamento de redundância. Conexões identificadas como pontes são candidatas imediatas a receberem rotas alternativas na próxima fase de expansão da infraestrutura, aumentando a resiliência da rede.

11. Bibliotecas Utilizadas e Conformidade Técnica

Biblioteca	Uso no Projeto
<code>pandas</code>	Exibição tabular de módulos e matriz de adjacência

Biblioteca	Uso no Projeto
<code>numpy</code>	Matriz de adjacência; regressão polinomial (<code>polyfit</code>)
<code>sympy</code>	Cálculo simbólico da derivada; resolução da equação de saturação
<code>heapq</code>	Fila de prioridade para o algoritmo de Dijkstra
<code>collections.Counter</code>	Contagem robusta de status dos módulos na análise ESG
<code>random</code>	Geração de dados operacionais simulados
<code>math, os</code>	Utilitários de sistema e matemáticos auxiliares

Todas as bibliotecas foram estudadas ao longo do curso e seu uso está em conformidade com os requisitos do projeto.

12. Conclusão

O SIGIC implementa de forma integrada os principais conteúdos da disciplina: grafos e algoritmos de rede (BFS, DFS, Dijkstra, Tarjan), estruturas de dados Python (listas, dicionários, tuplas, matrizes), modelagem matemática com cálculo diferencial (regressão quadrática e derivadas simbólicas) e análise de sustentabilidade e governança (ESG). A separação entre topologia estrutural e dados operacionais dinâmicos, aliada ao mecanismo de backup e restauração nas simulações de falha, garante um sistema robusto e extensível para o gerenciamento da colônia marciana Aurora Siger.